

Talk to Firmware - Introducción a la Ingeniería Inversa en Hardware con JTAG

Resolución de caja negra paso a paso

OpenOCD y GDB, volcado de memoria,
inspección hexadecimal y modificación controlada

Autores

Fabio Ruiz
Estefanny Villalta

Guía de resolución del reto: TALK // UNLOCK

1 Objetivo y escenario

El reto se resuelve como si se hubiera recibido únicamente una placa ESP32-C3 ya programada. No se supone acceso al proyecto con el que fue construida, a sus archivos de compilación ni a información sobre símbolos o nombres de funciones. El punto de partida es el hardware; cualquier imagen binaria utilizada debe extraerse primero del propio dispositivo.

La ruta de resolución utiliza únicamente:

1. observación del comportamiento físico;
2. reconocimiento de las protecciones;
3. extracción verificable de la flash;
4. control de la CPU y lectura cruda de memoria con OpenOCD y GDB;
5. comparación de volcados de RAM para encontrar un dato efímero;
6. inspección del binario extraído con GHex; y
7. un parche mínimo, verificable y reversible.

1.1 Resumen de la solución

En la primera fase, cada arranque produce una secuencia distinta de cuatro GPIO. Sin símbolos no puede pedirse a GDB la dirección por un nombre. En su lugar, se toman varias capturas completas de DRAM y se buscan ventanas de cuatro bytes que solo contengan GPIO válidos, no repitan valores y cambien entre ejecuciones.

La secuencia del último volcado se introduce físicamente y se confirma con GPIO9. El LED transmite el primer mensaje en Morse y la placa conserva el avance. Después se localiza `LOCKED` en el volcado de flash, se sustituye por `UNLOCK` en la copia de un único sector y se activa GPIO10. El LED transmite el segundo mensaje.

Resultado esperado

Los textos se obtienen de la observación y decodificación del LED:

`74LK 70 F1RMW4R3`

`4ND F1RMW4R3 W1LL 74LK 70 YOU`

En *leet speak*: `TALK TO FIRMWARE AND FIRMWARE WILL TALK TO YOU.`

2 Interfaz observable

Los datos iniciales proceden de la serigrafía y del diagrama de conexiones entregado con el taller.

Cuadro 1: Señales accesibles durante el reto.

Elemento	GPIO	Uso observable
LED	8	Indica estados y transmite código Morse.
VALIDAR	9	Confirma la secuencia de la primera fase.
AUXILIAR	10	Condición física de la segunda fase.
Candidatos	0, 1, 3, 4, 5, 6, 7	Pines que pueden integrar la secuencia.

- Conecte a GND un solo GPIO candidato a la vez.
- Mantenga cada contacto y cada liberación cerca de 100 ms. Es un margen práctico, no una afirmación sobre constantes internas.
- Si el LED indica rechazo, descarte la secuencia anterior.
- No reinicie después de leer la secuencia vigente en RAM.
- Observe el LED para decodificar el Morse. Como puede ser difícil analizar visualmente la secuencia, también se escribirá en la consola del monitor serial.

GDB

GDB puede detener y reanudar la CPU, consultar registros, leer memoria por dirección y guardar rangos crudos. La comparación entre varias capturas convierte esas operaciones básicas en una técnica de descubrimiento.

3 Herramientas

Cuadro 2: Herramientas utilizadas.

Herramienta	Uso
espefuse	Lectura no destructiva de eFuses.
esptool	Lectura y escritura controlada de flash.
OpenOCD	Servidor JTAG para el ESP32-C3.
riscv32-esp-elf-gdb	Depuración remota sin archivo objetivo ni símbolos.
GHex	Inspección y edición hexadecimal de copias binarias.
xxd, cmp, sha256sum	Verificación de bytes e integridad.
Python 3	Comparación reproducible de volcados de DRAM.

5 Nivel 1: adquisición de flash

5.1 Volcado completo

Determine primero el tamaño real. Los comandos usan 4 MiB porque corresponde a la placa del reto; debe ajustarse si `flash_id` informa otro valor.

```
# Entrar en la carpeta del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Crear la estructura de trabajo.
# evidencia/: copias originales obtenidas de la placa.
# solucion/: archivos derivados utilizados durante la resolución.
# solucion/ram/: capturas posteriores de DRAM.
mkdir -p evidencia solucion solucion/ram

# Consultar el tamaño y características de la memoria flash.
# En la placa del reto se espera una flash de 4 MiB.
esptool --chip esp32c3 --port /dev/ttyACM0 flash_id

# Leer los 4 MiB completos de la flash y guardar la adquisición original.
esptool --chip esp32c3 --port /dev/ttyACM0 \
    read_flash 0x0 0x400000 evidencia/dump_full.bin

# Calcular SHA-256 del volcado y guardar el hash como evidencia.
sha256sum evidencia/dump_full.bin | tee evidencia/SHA256SUMS

# Realizar una segunda lectura completa para comprobar la adquisición.
esptool --chip esp32c3 --port /dev/ttyACM0 \
    read_flash 0x0 0x400000 evidencia/dump_full-verify.bin

# Comparar byte a byte las dos adquisiciones.
# Si no imprime diferencias, los archivos son iguales.
cmp evidencia/dump_full.bin evidencia/dump_full-verify.bin

# Mostrar el código de salida del comando anterior.
# 0 = ambos archivos son idénticos.
echo "Comparison: $?"
```

```

Warning: Deprecated: Command 'flash_id' is deprecated. Use 'flash-id' instead.
esptool v5.3.1
Connected to ESP32-C3 on /dev/ttyACM0:
Chip type:      ESP32-C3 AZ (QFN32) (revision v1.1)
Features:      Wi-Fi, BT 5 (LE), Single Core, 160MHz, Embedded Flash 4MB (XMC)
Crystal frequency: 40MHz
USB mode:      USB-Serial/JTAG
MAC:          8c:fd:49:43:e7:c0

Stub flasher running.

Flash Memory Information:
=====
Manufacturer: 46
Device: 4016
Detected flash size: 4MB

Hard resetting via RTS pin...
Warning: Deprecated: Command 'read_flash' is deprecated. Use 'read-flash' instead.
esptool v5.3.1
Connected to ESP32-C3 on /dev/ttyACM0:
Chip type:      ESP32-C3 AZ (QFN32) (revision v1.1)
Features:      Wi-Fi, BT 5 (LE), Single Core, 160MHz, Embedded Flash 4MB (XMC)
Crystal frequency: 40MHz
USB mode:      USB-Serial/JTAG
MAC:          8c:fd:49:43:e7:c0

Stub flasher running.

Configuring flash size...
Read 4194304 bytes from 0x00000000 in 15.9 seconds (2108.9 kbit/s) to 'evidencia/dump_full.bin'.

Hard resetting via RTS pin...
766721cfce12452142b204186794261ec49bc383dbac554cfb7e3c2fe3835111 evidencia/dump_full.bin
Warning: Deprecated: Command 'read_flash' is deprecated. Use 'read-flash' instead.
esptool v5.3.1
Connected to ESP32-C3 on /dev/ttyACM0:
Chip type:      ESP32-C3 AZ (QFN32) (revision v1.1)
Features:      Wi-Fi, BT 5 (LE), Single Core, 160MHz, Embedded Flash 4MB (XMC)
Crystal frequency: 40MHz
USB mode:      USB-Serial/JTAG
MAC:          8c:fd:49:43:e7:c0

Stub flasher running.

Configuring flash size...
Read 4194304 bytes from 0x00000000 in 15.8 seconds (2120.4 kbit/s) to 'evidencia/dump_full-verify.bin'.

Hard resetting via RTS pin...
Comparison: 0
fabio@DELL:~/Descargas/Talk_to_Firmware_v2$

```

Figura 2: Dos lecturas idénticas de flash y extracción desde el volcado.

5.2 Tabla de particiones

Extraiga y decodifique la tabla desde la placa:

```

# Extraer la tabla de particiones desde el offset estándar 0x8000.
esptool --chip esp32c3 --port /dev/ttyACM0 \
    read_flash 0x8000 0x1000 solucion/partition_table.bin

# Decodificar la tabla binaria utilizando la herramienta de ESP-IDF
# y guardar también una copia en texto.

```

```
python3 "$IDF_PATH/components/partition_table/gen_esp32part.py" \
    solucion/partition_table.bin | tee solucion/partition_table.txt
```

```
fabio@DELL:~/Descargas/Talk_to_Firmware_v2$ # Extraer la tabla de particiones desde el offset estándar 0x8000.
esptool --chip esp32c3 --port /dev/ttyACM0 \
read_flash 0x8000 0x1000 solucion/partition_table.bin
# Decodificar la tabla binaria utilizando la herramienta de ESP-IDF
# y guardar también una copia en texto.
python3 "$IDF_PATH/components/partition_table/gen_esp32part.py" \
solucion/partition_table.bin | tee solucion/partition_table.txt
Warning: Deprecated: Command 'read_flash' is deprecated. Use 'read-flash' instead.
esptool v5.3.1
Connected to ESP32-C3 on /dev/ttyACM0:
Chip type:      ESP32-C3 AZ (QFN32) (revision v1.1)
Features:       Wi-Fi, BT 5 (LE), Single Core, 160MHz, Embedded Flash 4MB (XMC)
Crystal frequency: 40MHz
USB mode:       USB-Serial/JTAG
MAC:            8c:fd:49:43:e7:c0

Stub flasher running.

Configuring flash size...
Read 4096 bytes from 0x00008000 in 0.0 seconds (2192.9 kbit/s) to 'solucion/partition_table.bin'.

Hard resetting via RTS pin...
Parsing binary partition input...
Verifying table...
# ESP-IDF Partition Table
# Name, Type, SubType, Offset, Size, Flags
nvs,data,nvs,0x9000,24K,
factory,app,factory,0x10000,1M,
fabio@DELL:~/Descargas/Talk_to_Firmware_v2$
```

Figura 3: Extracción de la tabla de particiones.

En la unidad del taller se observan, entre otras, estas entradas:

```
nvs,      data, nvs,      0x9000,  0x24000
factory,  app,  factory, 0x10000, 0x100000
```

Confirme su propia tabla antes de reutilizar estos offsets. Después extraiga las copias de trabajo:

```
# Extraer la partición NVS desde el dump original.
# Offset: 0x9000.
# Tamaño: 0x24000 según la tabla de particiones mostrada.
dd if=evidencia/dump_full.bin of=evidencia/nvs-original.bin \
    bs=1 skip=$((0x9000)) count=$((0x24000)) status=none

# Extraer la partición factory como copia de trabajo.
# Comienza en 0x10000 y ocupa 0x100000 bytes (1 MiB).
dd if=evidencia/dump_full.bin of=solucion/factory.bin \
    bs=1 skip=$((0x10000)) count=$((0x100000)) status=none

# Calcular hashes de ambas extracciones para poder identificarlas
# y detectar modificaciones posteriores.
sha256sum evidencia/nvs-original.bin solucion/factory.bin
```

```
• fabio@DELL:~/Descargas/Talk_to_Firmware_v2$ # Extraer la partición NVS desde el dump original.
# Offset: 0x9000.
# Tamaño: 0x24000 según la tabla de particiones mostrada.
dd if=evidencia/dump_full.bin of=evidencia/nvs-original.bin \
    bs=1 skip=$((0x9000)) count=$((0x24000)) status=none

# Extraer la partición factory como copia de trabajo.
# Comienza en 0x10000 y ocupa 0x100000 bytes (1 MiB).
dd if=evidencia/dump_full.bin of=solucion/factory.bin \
    bs=1 skip=$((0x10000)) count=$((0x100000)) status=none

# Calcular hashes de ambas extracciones para poder identificarlas
# y detectar modificaciones posteriores.
sha256sum evidencia/nvs-original.bin solucion/factory.bin
ae8ebc98fd14dd6e15c04bfddb86c89f23be50e1f3347ca5b7b90812c9088bf3  evidencia/nvs-original.bin
43da2e12edd228e6fb4abe5bcf27741cac26161d5bd5ac01578a4e0edc8186c1  solucion/factory.bin
○ fabio@DELL:~/Descargas/Talk_to_Firmware_v2$
```

Figura 4: Ejecución de los comandos anteriores.

6 Nivel 2: OpenOCD y GDB

El objetivo es localizar un dato pequeño que cambia en cada ejecución: cuatro números de GPIO distintos. Esa restricción permite encontrarlo.

6.1 OpenOCD

En una terminal en VS Code, entre en la carpeta del reto y active el entorno de ESP-IDF. OpenOCD utilizará la interfaz USB/JTAG integrada del ESP32-C3 y abrirá el servidor al que posteriormente se conectará GDB.

```
# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Activar las herramientas de ESP-IDF 6.0.1.
source ~/.espressif/v6.0.1/esp-idf/export.sh

# Iniciar OpenOCD utilizando el USB/JTAG integrado del ESP32-C3.
# GDB se conectará posteriormente al servidor que escucha en el puerto 3333.
openocd -f board/esp32c3-builtin.cfg

# Salida esperada:
# Info : JTAG tap: esp32c3.cpu tap/device found
# Info : starting gdb server for esp32c3 on 3333
```

```

fabio@DELL:~/Descargas/Talk_to_Firmware_v2$ # Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2
# Activar las herramientas de ESP-IDF 6.0.1.
source ~/.espressif/v6.0.1/esp-idf/export.sh
# Iniciar OpenOCD utilizando el USB/JTAG integrado del ESP32-C3.
# GDB se conectará posteriormente al servidor que escucha en el puerto 3333.
openocd -f board/esp32c3-builtin.cfg
# Salida esperada:
# Info : JTAG tap: esp32c3.cpu tap/device found
# Info : starting gdb server for esp32c3 on 3333
Checking "python3" ...
Python 3.13.5
"python3" has been detected
Activating ESP-IDF 6.0
Setting IDF_PATH to '/home/fabio/.espressif/v6.0.1/esp-idf'.
* Checking python version ... 3.13.5
* Checking python dependencies ... OK
* Deactivating the current ESP-IDF environment (if any) ... OK
* Establishing a new ESP-IDF environment ... OK
* Identifying shell ... bash
* Detecting outdated tools in system ... Found tools that are not used by active ESP-IDF version.
For removing old versions of components, python use command 'python /home/fabio/.espressif/v6.0.1/esp-idf/tools/idf_tools.py uninstall'
To free up even more space, remove installation packages of those tools.
Use option python /home/fabio/.espressif/v6.0.1/esp-idf/tools/idf_tools.py uninstall --remove-archives.
* Shell completion ... Autocompletion code generated

Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

idf.py build
Open On-Chip Debugger v0.12.0-esp32-20260304 (2026-03-04-20:07)
Licensed under GNU GPL v2
For bug reports, read
  http://openocd.org/doc/doxygen/bugs.html
Info : esp_usb_jtag: VID set to 0x303a and PID to 0x1001
Info : esp_usb_jtag: capabilities descriptor set to 0x2000
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : esp_usb_jtag: serial (8C:FD:49:43:E7:C0)
Info : esp_usb_jtag: Device found. Base speed 40000KHz, div range 1 to 255
Info : clock speed 40000 kHz
Info : JTAG tap: esp32c3.tap0 tap/device found: 0x00005c25 (mfg: 0x612 (Espressif Systems), part: 0x0005, ver: 0x0)
Info : [esp32c3] datacount=2 progbufsize=16
Info : [esp32c3] Examined RISC-V core
Info : [esp32c3] XLEN=32, misa=0x40101104
Info : [esp32c3] Examination succeed
Info : [esp32c3] starting gdb server on 3333
Info : Listening on port 3333 for gdb connections
[]

```

Figura 5: OpenOCD detectando el ESP32-C3 y abriendo el puerto 3333.

6.2 GDB sin archivo objetivo

En una segunda terminal se inicia GDB. GDB se conecta directamente con OpenOCD y permite controlar e inspeccionar el procesador.

```

# Entrar en la misma carpeta utilizada para la resolución.
cd ~/Descargas/Talk_to_Firmware_v2

# Activar las herramientas RISC-V de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh

# Iniciar GDB.
riscv32-esp-elf-gdb

```

Con GDB abierto, conecte el depurador con OpenOCD y compruebe que existe control sobre el procesador. El desensamblado no tendrá nombres de funciones ni líneas de código fuente, lo cual es esperado en este escenario.

```

# Evitar que GDB pause salidas largas.
(gdb) set pagination off

```

```
# Conectar con el servidor GDB iniciado por OpenOCD.  
(gdb) target extended-remote localhost:3333  
  
# Reiniciar el ESP32-C3 y dejar la CPU detenida.  
(gdb) monitor reset halt  
  
# Mostrar el contenido de los registros del procesador.  
(gdb) info registers  
  
# Desensamblar ocho instrucciones a partir del PC actual.  
(gdb) x/8i $pc
```

El desensamblado aparece sin nombres ni líneas. Solo confirma que JTAG funciona.

```

fabio@DELL:~/Descargas/Talk_to_Firmware_v2$ # Entrar en la misma carpeta utilizada para la resolución.
cd ~/Descargas/Talk_to_Firmware_v2
# Activar las herramientas RISC-V de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh
# Iniciar GDB. ...
"python3" has been detected
Activating ESP-IDF 6.0
Setting IDF_PATH to '/home/fabio/.espressif/v6.0.1/esp-idf'.
* Checking python version ... 3.13.5
* Checking python dependencies ... OK
* Deactivating the current ESP-IDF environment (if any) ... OK
* Establishing a new ESP-IDF environment ... OK
* Identifying shell ... bash
* Detecting outdated tools in system ... Found tools that are not used by active ESP-IDF version.
For removing old versions of components, python use command 'python /home/fabio/.espressif/v6.0.1/esp-idf/tools/idf_tools.py uninstall'
To free up even more space, remove installation packages of those tools.
Use option python /home/fabio/.espressif/v6.0.1/esp-idf/tools/idf_tools.py uninstall --remove-archives.
* Shell completion ... Autocompletion code generated

Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

idf.py build
GNU gdb (esp-gdb) 16.3_20250913
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-linux-gnu --target=riscv32-esp-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) set pagination off
(gdb) target extended-remote localhost:3333
Remote debugging using localhost:3333
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x40000000 in ?? ()
(gdb) monitor reset halt
JTAG tap: esp32c3.tap0 tap/device found: 0x00005c25 (mfg: 0x612 (Espressif Systems), part: 0x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core reset)
(gdb) info registers
ra          0x0          0x0
sp          0x0          0x0
gp          0x0          0x0
tp          0x0          0x0
t0          0x0          0
t1          0x0          0
t2          0x0          0
fp          0x0          0x0
s1          0x0          0
a0          0x0          0
a1          0x0          0
a2          0x0          0
a3          0x0          0
a4          0x0          0
a5          0x0          0
a6          0x0          0
a7          0x0          0
s2          0x0          0
s3          0x0          0
s4          0x0          0
s5          0x0          0
s6          0x0          0
s7          0x0          0
s8          0x0          0
s9          0x0          0
s10         0x0          0
s11         0x0          0
t3          0x0          0
t4          0x0          0
t5          0x0          0
t6          0x0          0
pc          0x40000000    0x40000000
(gdb) x/8i $pc
=> 0x40000000: j          0x40002090
0x40000004: unimp
0x40000006: unimp
0x40000008: unimp
0x4000000a: unimp
0x4000000c: unimp
0x4000000e: unimp
0x40000010: c.nop    1
(gdb)

```

Figura 6: GDB conectado y con acceso a los registros.

6.3 Cuatro capturas de DRAM

La SRAM accesible por el bus de datos se encuentra entre 0x3FC80000 y 0x3FCDFFFF. Como el segundo argumento final de `dump binary memory` es exclusivo, se utiliza 0x3FCE0000. Se obtienen cuatro capturas independientes y se almacenan en `solucion/ram/` para poder compararlas posteriormente.

```
# Primera ejecución: reiniciar y dejar correr el firmware durante 2 segundos.
(gdb) monitor reset run
(gdb) shell sleep 2
```

```
# Detener la CPU para obtener una imagen consistente de la RAM.
(gdb) monitor halt
```

```
# Guardar la primera captura en solucion/ram/.
(gdb) dump binary memory solucion/ram/ram-01.bin 0x3FC80000 0x3FCE0000
```

```
# Repetir el procedimiento para una segunda ejecución.
(gdb) monitor reset run
(gdb) shell sleep 2
(gdb) monitor halt
(gdb) dump binary memory solucion/ram/ram-02.bin 0x3FC80000 0x3FCE0000
```

```
# Tercera captura.
(gdb) monitor reset run
(gdb) shell sleep 2
(gdb) monitor halt
(gdb) dump binary memory solucion/ram/ram-03.bin 0x3FC80000 0x3FCE0000
```

```
# Cuarta captura.
(gdb) monitor reset run
(gdb) shell sleep 2
(gdb) monitor halt
(gdb) dump binary memory solucion/ram/ram-04.bin 0x3FC80000 0x3FCE0000
```

Después del cuarto volcado no se debe ejecutar otro `reset` ni `continue`. La cuarta captura corresponde al estado que permanece detenido en GDB y que se utilizará posteriormente para validar la secuencia encontrada.

Si el rango completo no puede leerse

Confirme primero que la CPU esté detenida. Si OpenOCD informa una zona ilegible, divida el rango en bloques y conserve la dirección base de cada archivo. Use direcciones procedentes de una compilación propia.

```
(gdb) monitor reset run
JTAG tap: esp32c3.tap0 tap/device found: 0x00005c25 (mfg: 0x612 (Espressif Systems), part: 0x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core reset)
(gdb) shell sleep 2
(gdb) monitor halt
[esp32c3] Target halted, PC=0x40385236, debug_reason=00000000
(gdb) dump binary memory solucion/ram/ram-01.bin 0x3FC80000 0x3FCE0000
(gdb) monitor reset run
JTAG tap: esp32c3.tap0 tap/device found: 0x00005c25 (mfg: 0x612 (Espressif Systems), part: 0x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core reset)
(gdb) shell sleep 2
(gdb) monitor halt
[esp32c3] Target halted, PC=0x40383842, debug_reason=00000000
(gdb) dump binary memory solucion/ram/ram-02.bin 0x3FC80000 0x3FCE0000
(gdb) monitor reset run
JTAG tap: esp32c3.tap0 tap/device found: 0x00005c25 (mfg: 0x612 (Espressif Systems), part: 0x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core reset)
(gdb) shell sleep 2
(gdb) monitor halt
[esp32c3] Target halted, PC=0x420115FE, debug_reason=00000000
(gdb) dump binary memory solucion/ram/ram-03.bin 0x3FC80000 0x3FCE0000
(gdb) monitor reset run
JTAG tap: esp32c3.tap0 tap/device found: 0x00005c25 (mfg: 0x612 (Espressif Systems), part: 0x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core reset)
(gdb) shell sleep 2
(gdb) monitor halt
[esp32c3] Target halted, PC=0x40383978, debug_reason=00000000
(gdb) dump binary memory solucion/ram/ram-04.bin 0x3FC80000 0x3FCE0000
(gdb) □
```

Figura 7: Ejecución de la 4 capturas de RAM.

6.4 Filtro diferencial

Guarde el siguiente programa como `solucion/buscar_secuencia.py`. El script recorre las cuatro capturas de RAM y busca posiciones que contengan cuatro GPIO distintos pertenecientes al conjunto permitido. Además, exige que el contenido cambie entre ejecuciones para descartar patrones estáticos presentes accidentalmente en memoria.

```
from pathlib import Path

# Dirección inicial utilizada al realizar los dumps de DRAM.
BASE = 0x3FC80000

# GPIO que pueden formar parte de la secuencia.
ALLOWED = {0, 1, 3, 4, 5, 6, 7}

# Cargar todas las capturas almacenadas en solucion/ram/.
paths = sorted(Path("solucion/ram").glob("ram-*.bin"))

# Se necesitan al menos cuatro ejecuciones para el filtro diferencial.
if len(paths) < 4:
    raise SystemExit("At least four dumps are required")

# Leer cada archivo completo en memoria.
blobs = [path.read_bytes() for path in paths]

# Todos los dumps deben tener exactamente el mismo tamaño.
size = len(blobs[0])
if any(len(blob) != size for blob in blobs):
    raise SystemExit("All dumps must have the same size")
```

```

candidates = []

# Recorrer cada posible ventana de cuatro bytes.
for offset in range(size - 3):

    # Leer la misma posición en cada captura.
    samples = [
        tuple(blob[offset:offset + 4])
        for blob in blobs
    ]

    # Cada muestra debe contener cuatro GPIO válidos y distintos.
    valid = all(
        len(set(sample)) == 4 and set(sample) <= ALLOWED
        for sample in samples
    )

    # La tupla también debe variar entre ejecuciones.
    if valid and len(set(samples)) > 1:
        candidates.append((BASE + offset, samples))

# Mostrar las direcciones candidatas encontradas.
for address, samples in candidates:
    print(f"{address:#010x}: {samples}")

# Avisar cuando el filtro no encuentre ninguna coincidencia.
if not candidates:
    print("No candidates; collect more dumps and try again")

```

Desde una tercera terminal, situada en la carpeta principal del reto, ejecute el filtro y conserve simultáneamente su salida en `solucion/candidatos-ram.txt`.

```

# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Ejecutar el análisis de las cuatro capturas.
# tee permite mostrar la salida y guardarla al mismo tiempo.
python3 solucion/buscar_secuencia.py | \
    tee solucion/candidatos-ram.txt

```

El filtro exige cuatro bytes distintos, pertenecientes al conjunto permitido, y una tupla variable entre arranques. Así se descartan tablas constantes que también puedan contener números de GPIO. Una salida ilustrativa sería:

```

0x3fcXXXXX: [(5, 3, 7, 1), (0, 6, 4, 3),
              (7, 1, 5, 0), (3, 6, 1, 5)]

```

La dirección debe proceder de la placa analizada. Si quedan varios candidatos, tome más capturas. Si no aparece ninguno, repita con un tiempo de espera mayor y controlado.

6.5 Confirmación y entrada física

El resultado del filtro proporciona una dirección candidata dentro de DRAM. Antes de utilizar los GPIO físicamente, se comprueba esa misma dirección directamente sobre la cuarta ejecución, que continúa detenida en GDB. Los cuatro valores deben coincidir con la última tupla correspondiente a `ram-04.bin`.

```
# Examinar cuatro bytes sin signo desde la dirección encontrada.
# Sustituya 0xDIRECCION_CANDIDATA por el valor producido por el script.
(gdb) x/4ub 0xDIRECCION_CANDIDATA

# Reanudar exactamente esta ejecución, sin efectuar otro reset.
(gdb) continue
```

Debe coincidir con la última tupla de `ram-04.bin`. Anótela y reanude sin reiniciar:

Introduzca los cuatro GPIO en el orden mostrado por GDB. Cada GPIO se conecta individualmente a GND y se libera completamente antes de pasar al siguiente. Una vez introducidos los cuatro contactos, GPIO9 se lleva brevemente a GND para confirmar la respuesta. Si el firmware rechaza el intento y emite SOS, la secuencia deja de considerarse válida y debe repetirse el procedimiento de adquisición.

Antes de reanudar la ejecución puede abrirse una terminal adicional para registrar la salida serial del firmware. El monitor debe abrirse sin provocar un reinicio, ya que la secuencia obtenida de RAM corresponde exactamente a la ejecución que permanece detenida en GDB.

```
# Identificar el puerto asignado actualmente a la placa.
ls -l /dev/ttyACM*

# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Activar el entorno de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh

# Abrir ESP-IDF Monitor sin reiniciar la placa.
# La salida aparecera conforme el firmware la transmita.
idf.py -p /dev/ttyACM0 monitor --no-reset
```

Con el monitor abierto, vuelva a la terminal de GDB y reanude la misma ejecución:

```
# Reanudar la ejecucion detenida sin efectuar un nuevo reset.
(gdb) continue
```

Introduzca entonces los cuatro GPIO en el orden obtenido y confirme la secuencia mediante GPIO9. Si la respuesta es correcta, el LED emitirá el primer mensaje en Morse y su representación textual aparecerá simultáneamente en el monitor serie.

Resultado esperado

El primer patrón luminoso decodifica como:

74LK 70 F1RMW4R3

```

Entrar en la misma carpeta utilizada para la resolución.
cd ~/Descargas/Talk_to_Firmware_v2
# Activar las herramientas RISC-V de ESP-IDF.
...
reason=00000000
(gdb) dump binary memory solucion/ram/ram-02.b
in 0x3FC80000 0x3FCE0000
(gdb) monitor reset run
JTAG tap: esp32c3.tap0 tap/device found: 0x000
05c25 (mfg: 0x612 (Espressif Systems), part: 0
x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core res
et)
(gdb) shell sleep 2
(gdb) monitor halt
[esp32c3] Target halted, PC=0x420115FE, debug_
reason=00000000
(gdb) dump binary memory solucion/ram/ram-03.b
in 0x3FC80000 0x3FCE0000
(gdb) monitor reset run
JTAG tap: esp32c3.tap0 tap/device found: 0x000
05c25 (mfg: 0x612 (Espressif Systems), part: 0
x0005, ver: 0x0)
[esp32c3] Reset cause (3) - (Software core res
et)
(gdb) shell sleep 2
(gdb) monitor halt
[esp32c3] Target halted, PC=0x40383978, debug_
reason=00000000
(gdb) dump binary memory solucion/ram/ram-04.b
in 0x3FC80000 0x3FCE0000
(gdb) x/4ub 0x3fc8bfc8
0x3fc8bfc8:      1      5      6      7
(gdb) continue
Continuing.

• fabio@DELL:~/Descargas/Talk_to_Firmware_v2$ #
Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2
# Ejecutar el análisis de las cuatro capturas.

# tee permite mostrar la salida y guardarla al
mismo tiempo.
python3 solucion/buscar_secuencia.py | \
tee solucion/candidatos-ram.txt
0x3fc8bfc8: [(0, 6, 7, 1), (6, 0, 4, 1), (1, 0
, 3, 4), (1, 5, 6, 7)]
fabio@DELL:~/Descarg
fabio@DELL:~/Descar
fabio@DELL:~/D
fabio@DELL:~/Descargas/Talk_to_Firmware_v2$

```

Figura 8: Dirección candidata y cuatro bytes de la ejecución vigente.

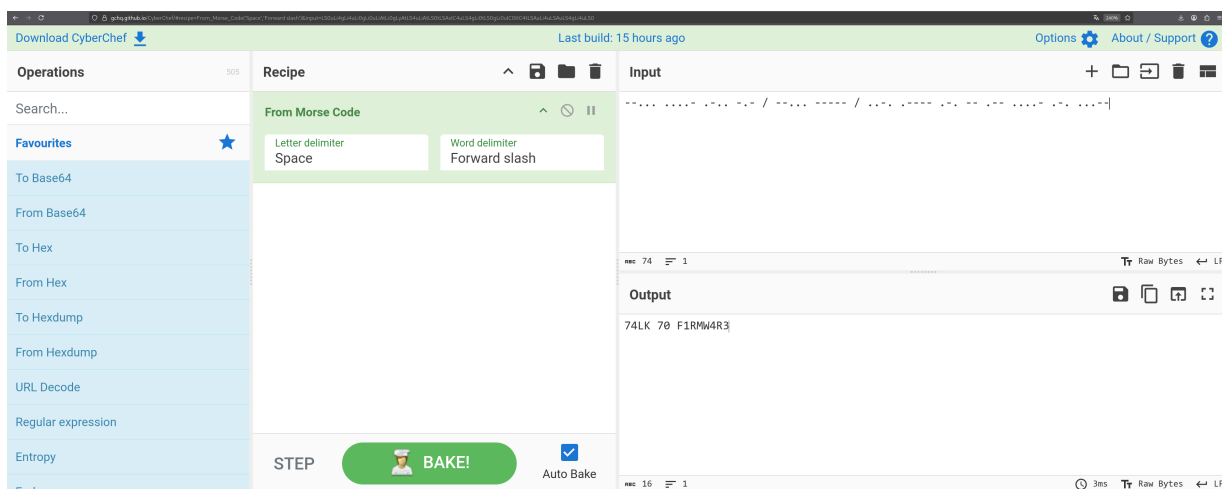


Figura 9: Registro y transcripción del primer mensaje del LED con ayuda de CyberChef (se copia la salida de puntos y rayas del monitor serial y se pega en CyberChef).

6.6 Persistencia observada

Después de completar correctamente la primera etapa, el firmware conserva un estado persistente. Para demostrarlo sin depender de nombres de claves ni de la estructura interna de NVS, se vuelve a extraer la misma partición y se compara byte a byte contra la copia original almacenada en `evidencia/`.

```
# Cerrar previamente GDB y OpenOCD para liberar el acceso a la placa.

# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Activar nuevamente las herramientas de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh

# Leer nuevamente la partición NVS.
# Offset: 0x9000.
# Tamaño: 0x24000 según la tabla de particiones obtenida anteriormente.
esptool --chip esp32c3 --port /dev/ttyACMO \
    read_flash 0x9000 0x24000 solucion/nvs-despues.bin

# Comparar la NVS posterior con la adquisición original.
# cmp no imprime nada si los archivos son idénticos.
cmp evidencia/nvs-original.bin solucion/nvs-despues.bin

# Mostrar el código de retorno de cmp.
# 0 = idénticos; cualquier valor distinto de 0 indica diferencias.
echo "NVS comparison: $?"
```

La diferencia demuestra que dentro del firmware cambió el estado y ahora ofrece la parte b del reto.

7 Nivel 3: revisión del binario con GHex

¿Qué es GHex?

GHex es un editor hexadecimal para Linux que permite inspeccionar y modificar el contenido binario de un archivo byte por byte.

Si GHex no se encuentra instalado, utilice el gestor de paquetes de su distribución:

```
# Debian, Ubuntu y derivados
sudo apt install ghex

# Arch Linux y derivados
sudo pacman -S ghex
```

7.1 Localización de LOCKED

Antes de modificar la flash se localiza la cadena `LOCKED` directamente sobre el volcado completo obtenido anteriormente. El objetivo es determinar su desplazamiento físico real.

El comando `grep` puede utilizarse sobre archivos binarios para buscar la secuencia de caracteres y mostrar el desplazamiento, en bytes, donde comienza la coincidencia.

```
# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Buscar la cadena LOCKED dentro del volcado completo.
#
# LC_ALL=C : trata los bytes de forma independiente de la configuracion regional.
# -a       : procesa el archivo binario como texto.
# -o       : muestra solamente la coincidencia encontrada.
# -b       : muestra el desplazamiento en bytes desde el inicio del archivo.
LC_ALL=C grep -aob 'LOCKED' evidencia/dump_full.bin
```

En la imagen utilizada para el reto aparece una coincidencia en el desplazamiento decimal 1048576. Al convertirlo a hexadecimal:

$$1048576_{10} = 0x100000.$$

Por tanto, la cadena comienza físicamente en la dirección 0x100000 de la flash.

Como la partición `factory` comienza físicamente en 0x10000, la misma posición expresada con respecto al comienzo de esa copia se encuentra en:

$$0x100000 - 0x10000 = 0xF0000.$$

Puede verificarse directamente sobre la copia de trabajo:

```
# Buscar LOCKED dentro de la copia de la particion factory.
# El desplazamiento esperado es 983040 decimal = 0xF0000.
LC_ALL=C grep -aob 'LOCKED' solucion/factory.bin
```

A continuación se puede inspeccionar cualquiera de las dos representaciones en GHex:

```
# Abrir el volcado completo.
# LOCKED se encuentra en 0x100000.
ghex evidencia/dump_full.bin

# Alternativamente, abrir solo la particion de aplicacion.
# En factory.bin la misma secuencia se encuentra en 0xF0000.
ghex solucion/factory.bin
```

El nombre del reto sugiere probar la sustitución `LOCKED` → `UNLOCK`. Sin embargo, en este punto sigue siendo únicamente una hipótesis. Su validez se comprobará mediante una modificación mínima y la observación posterior del comportamiento físico de la placa.

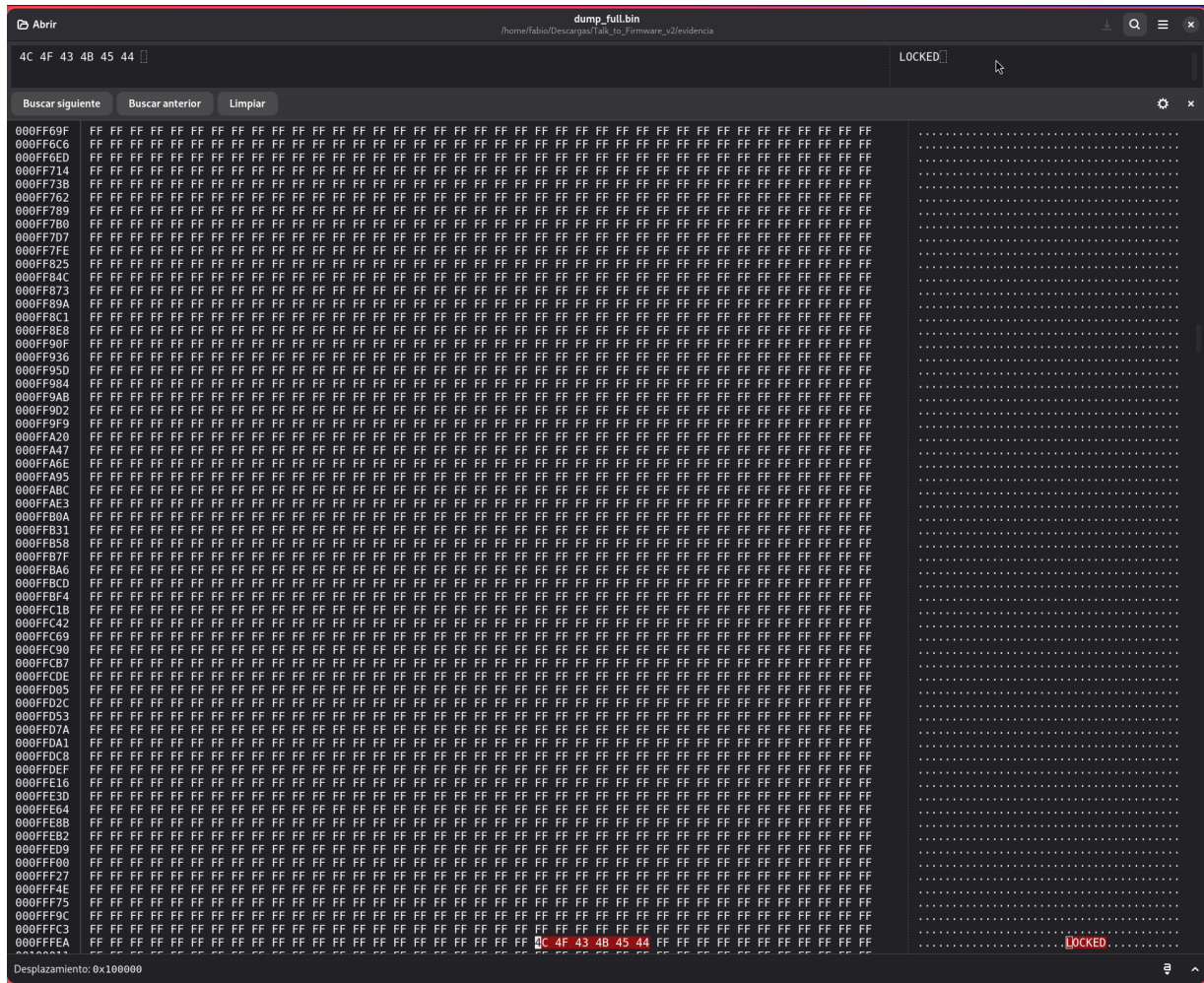


Figura 10: GHex mostrando la cadena LOCKED en la región identificada del binario.

7.2 Respaldo y edición de un sector

No es necesario reescribir toda la partición **factory** para modificar seis bytes. Antes de realizar cualquier cambio se extrae únicamente el sector de flash que contiene **LOCKED**. En esta placa se utiliza un sector de 0x1000 bytes, es decir, 4096 bytes.

El sector original se conserva dentro de **evidencia/**. A partir de él se crea una copia dentro de **solucion/**, que será la única que se modificará.

```
# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Activar nuevamente las herramientas de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh

# Leer 4096 bytes desde la direccion fisica 0x100000.
# El sector original se conserva como evidencia.
esptool --chip esp32c3 --port /dev/ttyACM0 \
  read_flash 0x100000 0x1000 evidencia/state-sector-original.bin
```

```
# Calcular el SHA-256 del sector original.
sha256sum evidencia/state-sector-original.bin

# Mostrar los primeros 16 bytes.
# Los seis primeros deben corresponder a LOCKED:
# 4c 4f 43 4b 45 44
xxd -g 1 -l 16 evidencia/state-sector-original.bin

# Crear una copia que si sera modificada.
cp evidencia/state-sector-original.bin \
  solucion/state-sector-unlock.bin

# Abrir solamente la copia de trabajo en GHex.
ghex solucion/state-sector-unlock.bin
```

En GHex se activa el modo de sobrescritura y se sustituyen exactamente los seis bytes:

LOCKED → UNLOCK

En hexadecimal:

4C 4F 43 4B 45 44 → 55 4E 4C 4F 43 4B

Ambas cadenas tienen exactamente seis caracteres, por lo que el cambio puede realizarse sin insertar ni eliminar bytes. De esta manera se conserva el tamaño del sector y permanecen intactas todas las posiciones posteriores.

Resultado esperado

```
00000000: 55 4e 4c 4f 43 4b ff ff ff ff ff ff ff ff  UNLOCK.....
```

Después de guardar el archivo con (ctrl + s) se comprueba que únicamente hayan cambiado los seis bytes esperados:

```
# Verificar que ambos archivos siguen midiendo exactamente 4096 bytes.
stat -c '%n: %s bytes' \
  evidencia/state-sector-original.bin \
  solucion/state-sector-unlock.bin

# Mostrar los primeros 16 bytes del sector modificado.
# Debe comenzar con 55 4e 4c 4f 43 4b = UNLOCK.
xxd -g 1 -l 16 solucion/state-sector-unlock.bin

# Comparar byte por byte el original contra la copia modificada.
# cmp -l muestra solamente las posiciones que son diferentes.
cmp -l evidencia/state-sector-original.bin \
  solucion/state-sector-unlock.bin
```

La comparación debe mostrar únicamente seis diferencias consecutivas al inicio del sector. Si aparecen diferencias adicionales, la copia debe descartarse y recrearse nuevamente desde el respaldo original.

```

fabio@ubuntu:~/Descargas/Talk_to_Firmware_v2$ cd ~/Descargas/Talk_to_Firmware_v2
fabio@ubuntu:~/Descargas/Talk_to_Firmware_v2$ source ~/.espressif/v6.0.1/esp-idf/export.sh
fabio@ubuntu:~/Descargas/Talk_to_Firmware_v2$ esptool --chip esp32c3 --port /dev/ttyACMO \
write_flash 0x100000 solucion/state-sector-unlock.bin
fabio@ubuntu:~/Descargas/Talk_to_Firmware_v2$ esptool --chip esp32c3 --port /dev/ttyACMO \
read_flash 0x100000 0x1000 solucion/state-sector-verify.bin
fabio@ubuntu:~/Descargas/Talk_to_Firmware_v2$ cmp solucion/state-sector-unlock.bin \
solucion/state-sector-verify.bin
fabio@ubuntu:~/Descargas/Talk_to_Firmware_v2$ echo "Write verification: $?"
Write verification: 0
fabio@ubuntu:~/Descargas/Talk_to_Firmware_v2$

```

Figura 11: Copia del sector mostrando UNLOCK sin alterar el tamaño del archivo.

7.3 Escritura y relectura

Una vez validada la copia modificada, se escribe nuevamente únicamente el sector localizado en 0x100000. No se vuelve a grabar toda la partición **factory**.

Inmediatamente después se realiza una lectura independiente de la misma región. La finalidad es comprobar que los bytes almacenados físicamente en la flash coinciden exactamente con el archivo que se pretendía escribir.

```

# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Activar nuevamente las herramientas de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh

# Escribir solamente el sector modificado en su direccion fisica original.
esptool --chip esp32c3 --port /dev/ttyACMO \
write_flash 0x100000 solucion/state-sector-unlock.bin

# Leer nuevamente los mismos 4096 bytes directamente desde la placa.
esptool --chip esp32c3 --port /dev/ttyACMO \
read_flash 0x100000 0x1000 solucion/state-sector-verify.bin

# Comparar la relectura contra el archivo que se escribio.
# cmp no muestra salida si los archivos son identicos.
cmp solucion/state-sector-unlock.bin \
solucion/state-sector-verify.bin

# Mostrar explicitamente el codigo de retorno.
# 0 significa que ambos archivos coinciden byte por byte.
echo "Write verification: $?"

# Comprobar visualmente los primeros 16 bytes releidos.
# Debe observarse UNLOCK al inicio.

```

```
xxd -g 1 -l 16 solucion/state-sector-verify.bin
```

Advertencia

No reescriba la partición completa para modificar únicamente seis bytes. El respaldo y la escritura de un solo sector reducen el alcance del cambio, facilitan la verificación byte a byte y permiten restaurar exactamente el contenido original si fuese necesario.

```
fabio@DELL:~/Descargas/Talk_to_Firmware_v2$ # Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2
# Activar nuevamente las herramientas de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh
# Escribir solamente el sector modificado en su direccion fisica original. ...
To free up even more space, remove installation packages of those tools.
Use option python /home/fabio/.espressif/v6.0.1/esp-idf/tools/idf_tools.py uninstall --remove-archives.
* Shell completion ... Autocompletion code generated

Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

idf.py build
Warning: Deprecated: Command 'write_flash' is deprecated. Use 'write-flash' instead.
esptool v5.3.1
Connected to ESP32-C3 on /dev/ttyACM0:
Chip type:      ESP32-C3 AZ (QFN32) (revision v1.1)
Features:      Wi-Fi, BT 5 (LE), Single Core, 160MHz, Embedded Flash 4MB (XMC)
Crystal frequency: 40MHz
USB mode:      USB-Serial/JTAG
MAC:          8c:fd:49:43:e7:c0

Stub flasher running.

Configuring flash size...
Flash will be erased from 0x00100000 to 0x00100fff...
Wrote 4096 bytes (35 compressed) at 0x00100000 in 0.0 seconds (6043.1 kbit/s).
Hash of data verified.

Hard resetting via RTS pin...
Warning: Deprecated: Command 'read_flash' is deprecated. Use 'read-flash' instead.
esptool v5.3.1
Connected to ESP32-C3 on /dev/ttyACM0:
Chip type:      ESP32-C3 AZ (QFN32) (revision v1.1)
Features:      Wi-Fi, BT 5 (LE), Single Core, 160MHz, Embedded Flash 4MB (XMC)
Crystal frequency: 40MHz
USB mode:      USB-Serial/JTAG
MAC:          8c:fd:49:43:e7:c0

Stub flasher running.

Configuring flash size...
Read 4096 bytes from 0x00100000 in 0.0 seconds (2157.2 kbit/s) to 'solucion/state-sector-verify.bin'.

Hard resetting via RTS pin...
fabio@DELL:~/Descargas/Talk_to_Firmware_v2$ # cmp no muestra salida si los archivos son identicos.
cmp solucion/state-sector-unlock.bin \
solucion/state-sector-verify.bin
# Mostrar explicitamente el codigo de retorno.
# 0 significa que ambos archivos coinciden byte por byte.
echo "Write verification: $?"
# Comprobar visualmente los primeros 16 bytes releidos.
# Debe observarse UNLOCK al inicio.
xxd -g 1 -l 16 solucion/state-sector-verify.bin
Write verification: 0
00000000: 55 4e 4c 4f 43 4b ff ff ff ff ff ff ff ff ff UNLOCK.....
fabio@DELL:~/Descargas/Talk_to_Firmware_v2$
```

Figura 12: Relectura independiente del sector modificado después de escribirlo en la flash.

8 Segunda fase y mensaje final

Después de completar la primera etapa existen tres condiciones observables que deben cumplirse simultáneamente:

1. la primera fase dejó un estado persistente en NVS;

2. la región de flash analizada contiene ahora UNLOCK;
3. GPIO10 se mantiene conectado a GND durante la comprobación.

La prueba final no requiere realizar nuevas modificaciones. Su objetivo es comprobar que el firmware reconoce simultáneamente las tres condiciones anteriores y entra en la segunda ruta de ejecución.

El procedimiento es el siguiente:

1. Cierre GDB y OpenOCD para evitar que interfieran con la ejecución normal.
2. Conecte GPIO10 a GND.
3. Abra ESP-IDF Monitor y manténgalo activo durante toda la prueba.
4. Reinicie la placa desde el propio monitor mediante **Ctrl+T** seguido de **Ctrl+R**.
5. Observe el banner y la representación textual del código Morse en la terminal.
6. Observe también el LED hasta que termine completamente la transmisión.
7. Registre los puntos, rayas y separaciones entre letras y palabras.

Para iniciar el monitor:

```
# Entrar en la carpeta principal del reto.
cd ~/Descargas/Talk_to_Firmware_v2

# Activar el entorno de ESP-IDF.
source ~/.espressif/v6.0.1/esp-idf/export.sh

# Abrir ESP-IDF Monitor sin provocar un reset automatico al conectarse.
idf.py -p /dev/ttyACM0 monitor --no-reset
```

Con el monitor abierto, no utilice el botón físico RESET/EN. En su lugar, reinicie la placa directamente desde ESP-IDF Monitor:

```
Ctrl+T
Ctrl+R
```

Ctrl+T activa el menú de comandos de ESP-IDF Monitor y **Ctrl+R** solicita el reinicio del ESP32-C3 mediante la línea RTS. De esta forma el monitor permanece preparado para recibir la salida desde el inicio de la nueva ejecución y se reduce el riesgo de perder el banner o los primeros símbolos del mensaje. ESP-IDF documenta este atajo como el mecanismo de reinicio del target desde el monitor.

Resultado esperado

El segundo patrón luminoso decodifica como:

4ND F1RMW4R3 W1LL 74LK 70 YOU

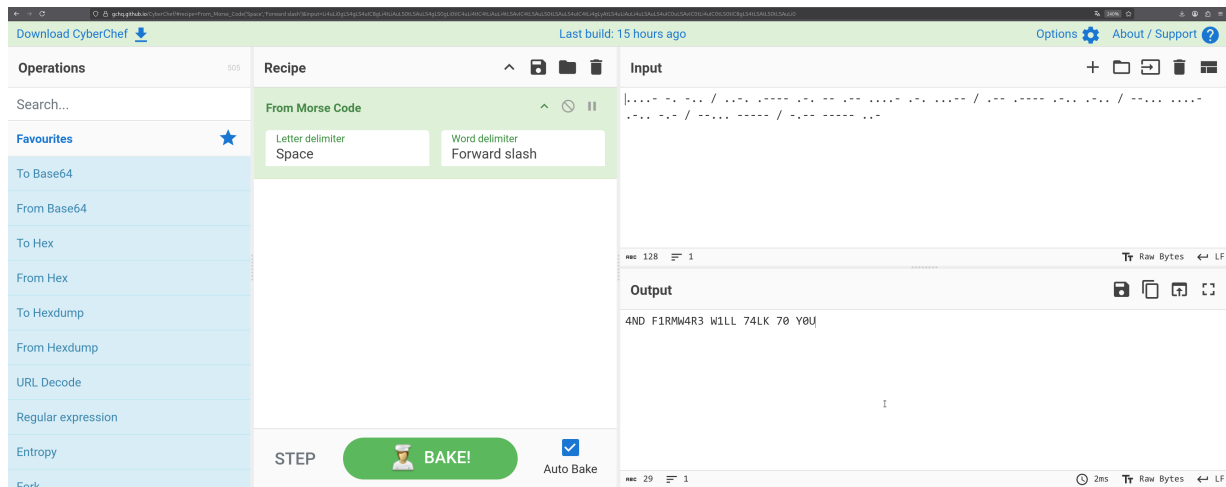


Figura 13: Registro y transcripción del segundo mensaje emitido por el LED y observado en el monitor serie.

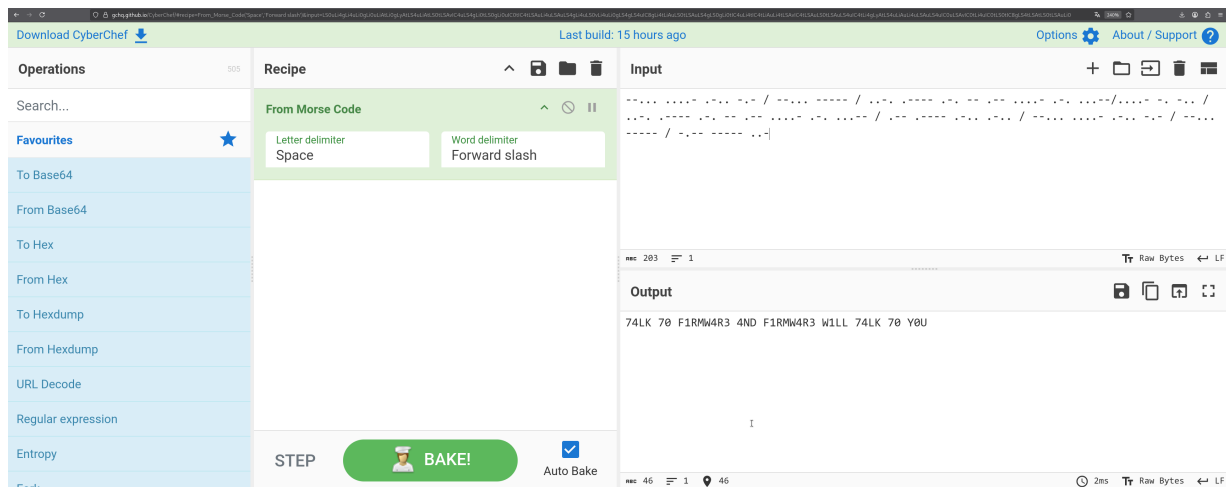


Figura 14: Registro y transcripción del mensaje completo emitido por el LED y observado en el monitor serie.

Fin del proceso de ingeniería inversa!

9 Referencias

Referencias

- [1] Espressif Systems, *ESP32-C3 Series Datasheet*. https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf
- [2] Espressif Systems, *ESP32-C3 Technical Reference Manual*. https://www.espressif.com/sites/default/files/documentation/esp32-c3_technical_reference_manual_en.pdf
- [3] Espressif Systems, *eFuse Manager – ESP32-C3*. ESP-IDF Programming Guide. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/api-reference/system/efuse.html>
- [4] Espressif Systems, *espefuse – ESP32-C3*. Esptool Documentation. <https://docs.espressif.com/projects/esptool/en/latest/esp32c3/espefuse/index.html>
- [5] Espressif Systems, *Esptool Documentation – ESP32-C3*. <https://docs.espressif.com/projects/esptool/en/latest/esp32c3/>
- [6] Espressif Systems, *Partition Tables – ESP32-C3*. ESP-IDF Programming Guide. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/api-guides/partition-tables.html>
- [7] Espressif Systems, *JTAG Debugging – ESP32-C3*. ESP-IDF Programming Guide. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/api-guides/jtag-debugging/>
- [8] Espressif Systems, *Configure ESP32-C3 Built-in JTAG Interface*. ESP-IDF Programming Guide. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/api-guides/jtag-debugging/configure-builtin-jtag.html>
- [9] Espressif Systems, *OpenOCD for ESP32*. <https://github.com/espressif/openocd-esp32>
- [10] Free Software Foundation, *Debugging with GDB: The GNU Source-Level Debugger*. <https://sourceware.org/gdb/current/onlinedocs/gdb.html>
- [11] Espressif Systems, *IDF Monitor*. ESP-IDF Programming Guide. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/api-guides/tools/idf-monitor.html>
- [12] The GNOME Project, *GHex – Hexadecimal Editor*. <https://gitlab.gnome.org/GNOME/ghex>